

Hybrid Sparse–Dense Retrieval for Privacy-Constrained Incident Resolution: Design and Evaluation of a Locally-Hosted Retrieval-Augmented Assistant

Ayoub Aarab

*Master Data Science — End-of-Year Project
Ecole Normale Supérieure, Martil*

July 16, 2026

Abstract

Retrieval-augmented generation (RAG) grounds a language model’s output in an external corpus, but its quality is bounded by the retrieval stage. In operational IT incident management, queries are heterogeneous: some are paraphrased descriptions of a symptom, while others are exact identifiers — incident numbers, error codes, function and table names. Dense embedding retrieval handles the former well and the latter poorly, since semantically near-identical identifier strings are not separable in embedding space.

This work designs and evaluates an incident-resolution assistant built on a hybrid retriever that fuses a dense sentence-embedding index with a sparse Okapi BM25 index using Reciprocal Rank Fusion (RRF). The system operates under a strict privacy constraint: all inference — embeddings and generation — executes on locally-hosted models, with no external API calls. The retriever is evaluated on a labeled benchmark of 272 queries in four stylistic classes over a corpus of 68 incident reports spanning nine technical domains, using Recall@ k , Mean Reciprocal Rank, and nDCG@5.

Hybrid retrieval attains Recall@5 = 1.000 and MRR = 0.987, against 0.768 and 0.678 for dense-only retrieval. A per-class analysis localizes the improvement: on exact-identifier queries, dense-only Recall@5 is 0.221 while the hybrid attains 1.000, confirming that the fusion supplies precisely the lexical precision the dense index lacks. Ablations show retrieval quality is invariant to chunk size over [400, 1000] characters and saturates at $k \geq 3$, a result used to bound the generation context. Error analysis records a 0.0% failure rate over the benchmark. The retrieval component is embedded in a grounded generation pipeline with intent-based abstention and adversarial-input defenses, and in an automated report-synthesis mechanism that maps a resolved dialogue onto a validated structured schema.

Contents

1	Introduction	3
1.1	Problem Setting	3
1.2	Deployment Constraint: Local Inference	3
1.3	Objectives and Structure	3
2	Background	4
2.1	Retrieval-Augmented Generation	4
2.2	Sparse Retrieval: Okapi BM25	4
2.3	Dense Retrieval	4
2.4	Rank Fusion	4
3	System Architecture	5
3.1	Design Overview	5
3.2	Structured Report Schema as a Shared Contract	5
3.3	Model Provider Abstraction	6
3.4	Deployment	6
4	Corpus, Representation, and Evaluation Protocol	6
4.1	Corpus	6
4.2	Document Representation	8
4.3	Chunking and Indexing	8
4.4	Evaluation Protocol	9
5	Retrieval Method	9
5.1	Components	9
5.2	Fusion	10
6	Experimental Results	10
6.1	Aggregate Retrieval Quality	10
6.2	Per-Class Analysis	11
6.3	Ablations	12
6.4	Error Analysis	12
7	Grounded Generation	13
7.1	Conditioning and Output Structure	13
7.2	Abstention Under Insufficient Evidence	13
7.3	Adversarial Input	14
7.4	Feedback	14
8	Report Synthesis	14
9	Discussion	15
9.1	Interpretation	15
9.2	Limitations	15
9.3	Future Work	16
9.4	Conclusion	16

1 Introduction

1.1 Problem Setting

Incident management in large-scale IT operations produces a corpus of structured post-mortem reports: each documents a symptom, a diagnosed root cause, a sequence of resolution steps, and supporting artifacts such as queries, commands, or configuration. This corpus is institutional knowledge. Its operational value depends entirely on whether an engineer facing a new incident can retrieve the relevant prior cases quickly and reliably.

Retrieval-augmented generation [4] is the natural formulation: retrieve the most relevant prior reports, condition a language model on them, and synthesize a grounded resolution. The scientific difficulty lies not in the generation stage but in retrieval. Incident queries are stylistically heterogeneous, and the two dominant retrieval paradigms fail on disjoint subsets of them:

- **Dense retrieval** embeds queries and documents into a shared metric space and ranks by cosine similarity. It captures paraphrase and semantic proximity, but identifiers such as INC0012017 and INC0012018 are lexically near-identical and semantically indistinguishable; the embedding space does not separate them.
- **Sparse retrieval** (Okapi BM25 [1]) ranks by weighted term overlap. It matches identifiers and rare technical tokens exactly, but is blind to synonymy and paraphrase.

An incident assistant must serve both query classes. This motivates a hybrid retriever, and it raises the central empirical question of this work: *how much does hybridization actually contribute, and on which queries does that contribution arise?*

1.2 Deployment Constraint: Local Inference

The system is subject to a hard privacy requirement: incident reports contain sensitive operational detail and must not be transmitted to third-party inference APIs. Consequently every model — the sentence encoder and the generative model — is self-hosted, and the pipeline performs no external network calls at query time.

This constraint is not incidental; it conditions the design. It excludes frontier hosted models, imposing a lower generation-quality ceiling and a hardware-bound latency floor. It therefore shifts the engineering leverage away from model scale and toward *context quality*: what is retrieved, and how it is presented to the model. This shift is the technical thesis of the work.

1.3 Objectives and Structure

The objectives are: (i) to design a retrieval pipeline that serves both semantic and lexical query classes under local-inference constraints; (ii) to quantify its behavior on a labeled benchmark with standard information-retrieval metrics and ablations; (iii) to embed it in a generation pipeline that abstains rather than fabricates when evidence is insufficient; and (iv) to close the documentation loop by synthesizing new structured reports from resolved dialogues.

Section 2 reviews the relevant retrieval background. Section 3 presents the system architecture. Section 4 describes the corpus, its representation, and the evaluation protocol.

Section 5 formalizes the retrieval method. Section 6 reports results. Sections 7–8 cover grounded generation, robustness, and report synthesis. Section 9 discusses limitations.

2 Background

2.1 Retrieval-Augmented Generation

RAG [4] conditions a generative model on documents retrieved from a non-parametric memory. Formally, given a query q , a retriever selects a set $D_q \subset \mathcal{D}$ from a corpus $\mathcal{D}$, and a generator produces an answer $a \sim p(a \mid q, D_q)$. Two properties make this attractive for incident resolution: the model’s output can be traced to cited evidence, and the corpus can be updated without retraining. The critical dependency is that $p(a \mid q, D_q)$ can only be as good as D_q : retrieval failure is unrecoverable downstream, since a model conditioned on irrelevant context either hallucinates or abstains.

2.2 Sparse Retrieval: Okapi BM25

BM25 [1] scores a document d against query q as

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}, \quad (1)$$

where $f(t, d)$ is the frequency of term t in d , $|d|$ the document length, avgdl the mean document length, and $\text{IDF}(t)$ the inverse document frequency. The parameter k_1 controls term-frequency saturation and b the strength of length normalization. The inverse-document-frequency factor assigns high weight to rare terms — exactly the regime of incident identifiers, error codes, and function names.

2.3 Dense Retrieval

Dense retrieval encodes text with a transformer bi-encoder trained so that semantically related pairs are close under cosine similarity [3]. Ranking is by

$$s_{\text{dense}}(q, d) = \frac{\phi(q) \cdot \phi(d)}{\|\phi(q)\| \|\phi(d)\|}, \quad (2)$$

with ϕ the encoder. This generalizes across surface form, but its inductive bias — mapping semantically similar text to nearby vectors — is precisely what makes it unable to discriminate between distinct identifiers that are semantically equivalent (all are “an incident number”).

2.4 Rank Fusion

Combining Eq. (1) and Eq. (2) by score arithmetic is ill-posed: the two are on incompatible, unnormalized scales, and any weighting requires per-corpus calibration. Reciprocal Rank Fusion [2] avoids this by discarding magnitudes and using only rank position:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad (3)$$

where R is the set of ranked lists and k a smoothing constant that damps the influence of top ranks. RRF is scale-free and effectively parameter-free, which is why it is adopted here.

3 System Architecture

3.1 Design Overview

The system is a single application exposing two functional modules over a shared corpus: an *assistant* that answers incident questions by retrieval-augmented generation, and a *report editor* that authors and maintains the structured reports that constitute the corpus. Both are served by one backend process and one web client.

The implementation is a modular monolith: a FastAPI backend with separated internal packages (retrieval and generation; report persistence; a shared schema and model-provider abstraction) and a React single-page client. A distributed decomposition is unwarranted — the modules share a corpus, target a single host, and have no independent scaling requirement — and would introduce network and operational overhead without benefit.

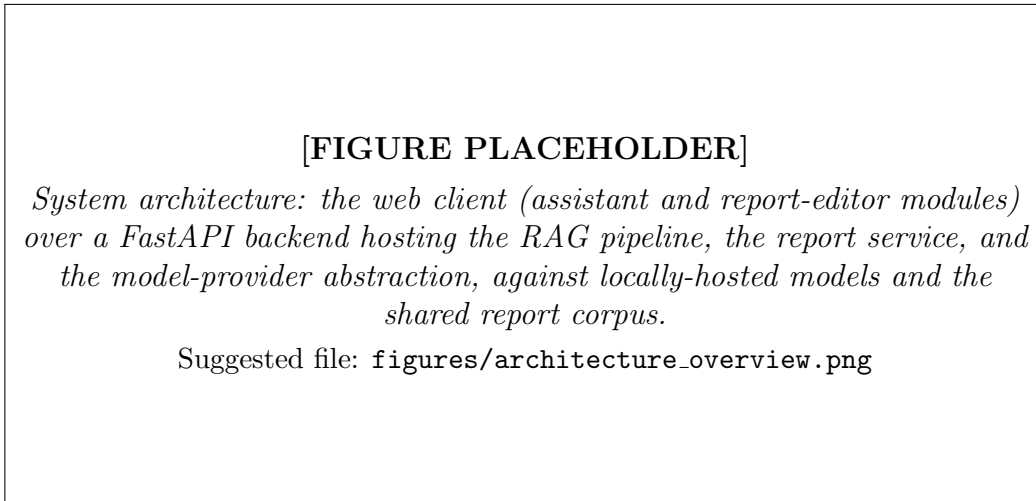


Figure 1: System architecture: the web client (assistant and report-editor modules) over a FastAPI backend hosting the RAG pipeline, the report service, and the model-provider abstraction, against locally-hosted models and the shared report corpus.

3.2 Structured Report Schema as a Shared Contract

Reports are represented by a typed schema, `Report = (metadata, blocks)`, defined once as Pydantic models and mirrored in the client’s type system. The `metadata` record carries identifying attributes (identifier, title, category, subcategory, reporter, date). The `blocks` field is a sequence over a discriminated union of content types: heading, paragraph, ordered/unordered list, code, table, image, and cross-referenced incident example.

This schema is the system’s central abstraction. It is simultaneously (i) the editor’s document model, (ii) the unit of ingestion for retrieval, and (iii) the output type of report synthesis (Section 8). A single validated contract across all three roles is what permits the assistant to both consume and produce corpus documents without translation layers.

3.3 Model Provider Abstraction

Generation is accessed through a provider interface exposing text and vision completion, with a locally-hosted implementation as the deployed backend. Decoupling the pipeline from a concrete model permits the generative model to be selected by configuration, and isolates the retrieval and orchestration logic from model-specific concerns. The deployed configuration uses a 3B-parameter instruction-tuned model for text and a 3B vision-language model for screenshot analysis; sentence embeddings use `all-MiniLM-L6-v2`, a 384-dimensional encoder.

3.4 Deployment

The application is containerized with a multi-stage build in which the client is compiled and served as static assets by the backend process, yielding a single-process deployment. Persistent state comprises the report corpus, the embedding-model cache, and a SQLite database holding dialogue history.

4 Corpus, Representation, and Evaluation Protocol

4.1 Corpus

The corpus comprises 68 incident reports spanning nine technical domains: networking, databases, authentication, infrastructure, frontend, messaging and queueing, continuous integration and delivery, monitoring, and security. Each report carries domain-characteristic failure signatures — HTTP status codes, message-broker error codes, container exit codes, TLS handshake alerts, DNS resolution failures.

The corpus is constructed to be discriminative rather than merely representative. It contains two families of near-miss pairs designed to defeat shallow matching:

- **Shared symptom, distinct root cause:** e.g. two incidents presenting as HTTP 503 from connection-pool exhaustion, one caused by a connection leak in an unrelated workload, the other by an undersized pool at peak concurrency.
- **Shared error code, distinct context:** e.g. container exit code 137 arising from node-level memory-pressure eviction versus a batch process exceeding its own memory limit.

A retriever that keyword-matches on 503 or 137 cannot separate these; correct ranking requires discriminating on context.

Table 1: Corpus statistics (source: evaluation notebook, cell 3).

Property	Value
Reports	68
Distinct categories	30
Median tokens per report	147.5
Token range per report	[103, 175]
Block-type frequency	
Paragraph	148
Heading	89
Code	72
List	68
Table	20
Incident example	12
Description box	8
Image	7

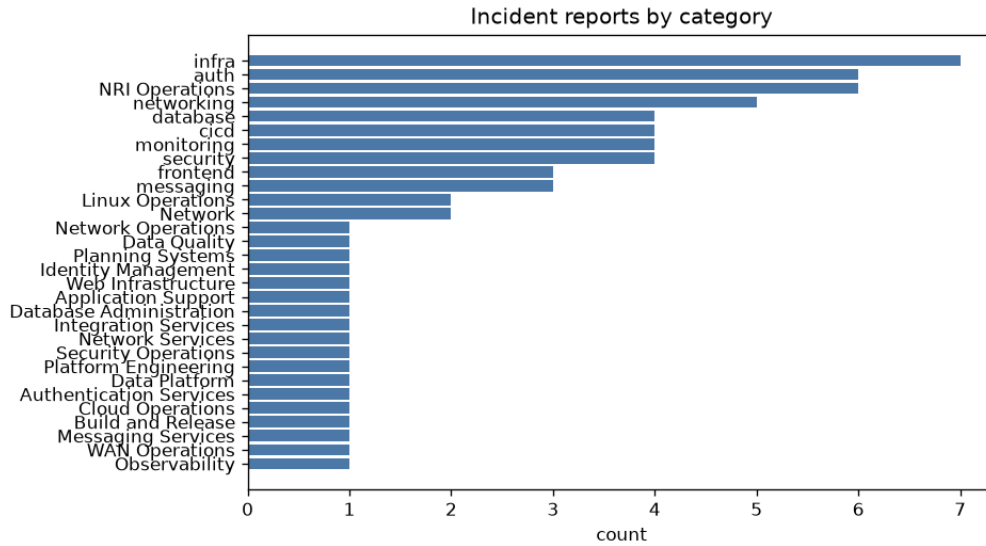


Figure 2: Category distribution over the corpus (source: notebook cell 3).

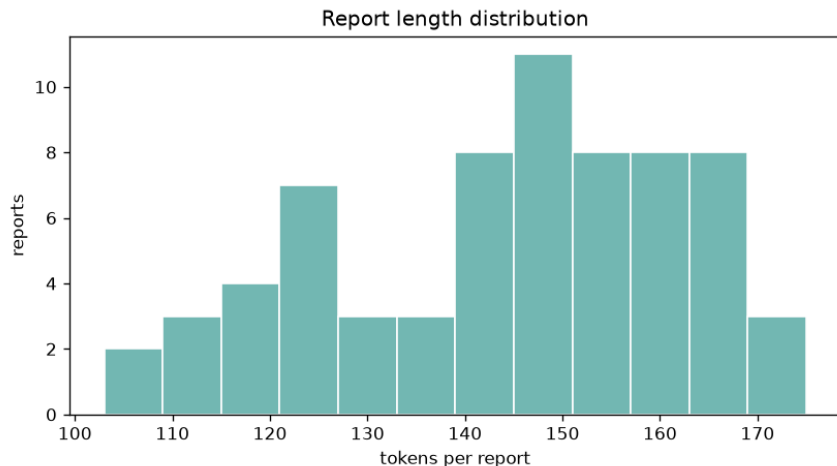


Figure 3: Report length distribution in tokens (source: notebook cell 3).

4.2 Document Representation

Reports are stored as structured documents, not free text, so the indexed representation must be derived by a projection $\pi : \text{Report} \rightarrow \Sigma^*$. The choice of π is consequential and easy to get wrong.

A naive projection serializes the document tree verbatim, emitting structural tokens — block identifiers, type discriminators, nesting keys, encoded image payloads — interleaved with content. The resulting string is dominated by tokens carrying no incident semantics. Both stages degrade: the encoder ϕ maps such strings into a region governed by shared structural vocabulary rather than incident content, collapsing inter-document distances; and BM25’s IDF weighting is diluted by high-frequency structural terms. Downstream, the generator is conditioned on text in which the actual symptom and resolution are a minority of tokens.

The system therefore defines π *schema-aware*: it dispatches on block type and emits only the semantic payload of each — heading text, symptom and root-cause prose with markup stripped, resolution list items, code content, table cells — while discarding identifiers, type tags, and binary data. Metadata is emitted as a leading header so that identifiers and category remain lexically searchable. This projection is a precondition for the retrieval results of Section 6: no amount of ranking refinement compensates for an uninformative representation.

4.3 Chunking and Indexing

Projected documents are segmented into overlapping character windows (700 characters, 120-character overlap) to bound the encoder’s input and to localize matching. Each chunk is encoded by ϕ into $\mathbb{R}^{384}$ and simultaneously tokenized into the BM25 index. Retrieval operates over chunks; results are aggregated to the report level by taking each source document’s maximum-scoring chunk, so a report is ranked by its single best-matching passage.

4.4 Evaluation Protocol

Evaluation requires query–document relevance labels. Absent production query logs, labels are constructed by inversion: for each report d , queries are synthesized that a user seeking d plausibly issues, with d as the gold document. Four stylistic classes are generated, chosen to span the semantic–lexical axis that motivates the hybrid:

- **title** — an interrogative form of the report title (semantic, high signal);
- **symptom** — the root-cause sentence (semantic, paraphrastic);
- **id** — the bare incident identifier (purely lexical);
- **keyword** — salient technical tokens such as function and table names (lexical, moderate).

This yields 272 labeled queries over the 68 reports. Stratifying by class is essential: an aggregate score would mask the fact that the two retrieval paradigms fail on *disjoint* query classes, which is the phenomenon under study.

Validity. The queries are synthetic and derived from the documents they retrieve, so the protocol measures recovery of a document from a paraphrase of its own content. Real queries are noisier, use vocabulary absent from the report, and may under-specify. The reported figures are therefore an upper bound on field performance. The protocol is nonetheless informative for its purpose here — it is a *comparative* instrument, and the dense/sparse/hybrid contrast is measured under identical conditions, so the relative ordering and the per-class localization of error are robust to this bias even where the absolute values are optimistic.

Metrics are Recall@ k (whether the gold report appears in the top k), Mean Reciprocal Rank, and nDCG@5, computed with a single relevant document per query.

5 Retrieval Method

5.1 Components

Three configurations are compared under identical chunking and identical corpus:

1. **Dense:** cosine ranking by Eq. (2) using `all-MiniLM-L6-v2`.
2. **Sparse:** BM25 by Eq. (1) with standard parameters $k_1 = 1.5$, $b = 0.75$. The index is implemented directly rather than by dependency, which keeps the retrieval stack self-contained and its behavior fully specified — relevant under a constraint that forbids external services. Tokenization preserves underscores, so composite identifiers such as `support_remove_opv_duplicate` remain single high-IDF terms rather than fragmenting into common subwords.
3. **Hybrid:** RRF over the dense and sparse rankings by Eq. (3), with $k = 60$.

5.2 Fusion

Each component produces a ranking over chunks. Documents scoring zero under BM25 are excluded from its list rather than appended arbitrarily, so that non-matching documents contribute no reciprocal-rank mass. The fused score of Eq. (3) is then aggregated per source report as described above, and the top K reports are returned.

Two parameters govern the interface with the generator. `TOP_K = 5` reports are retrieved and surfaced to the user as citations. Only the top `RESOLUTION_CONTEXT_K = 3` are placed in the generation context; Section 6.3 shows this choice is lossless with respect to retrieval coverage while materially reducing prompt length.

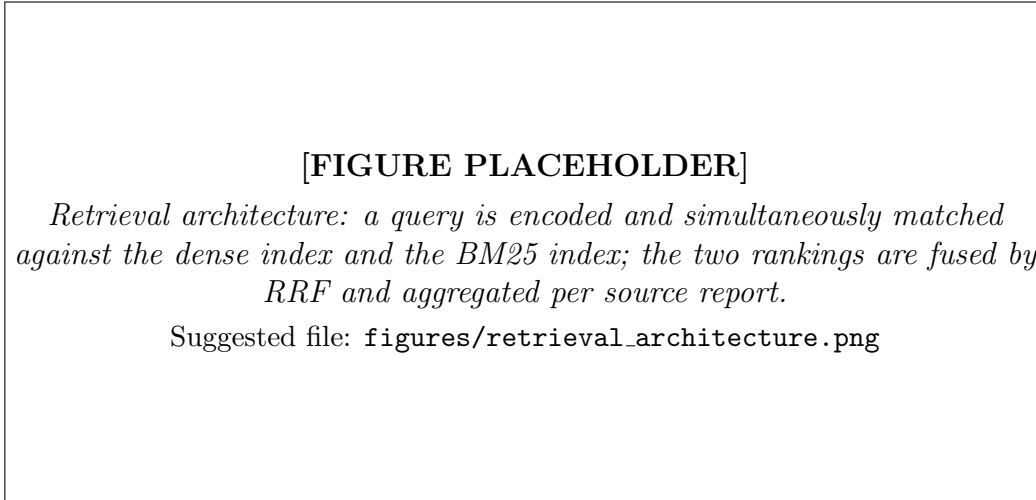


Figure 4: Retrieval architecture: a query is encoded and simultaneously matched against the dense index and the BM25 index; the two rankings are fused by RRF and aggregated per source report.

6 Experimental Results

All quantitative results are produced by the evaluation notebook (`eval/evaluation.ipynb`); each table cites its source cell.

6.1 Aggregate Retrieval Quality

Table 2 reports aggregate performance over the 272-query benchmark. The hybrid dominates both components on every metric, attaining $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense retrieval.

Table 2: Aggregate retrieval performance, 272 labeled queries (source: notebook cell 6).

Configuration	R@1	R@3	R@5	MRR	nDCG@5
Dense	0.629	0.713	0.768	0.678	0.700
Sparse (BM25)	0.779	0.982	0.989	0.872	0.902
Hybrid (RRF)	0.974	1.000	1.000	0.987	0.991

Two observations merit attention. First, sparse retrieval is the stronger single component on this corpus ($\text{Recall@5} = 0.989$ versus 0.768), which is consistent with a domain

whose documents are dense in rare, discriminative technical tokens; this is a corpus property, not a general claim about BM25. Second, and more informative, the hybrid’s advantage over BM25 is concentrated at the top rank: Recall@1 improves from 0.779 to 0.974 while Recall@5 improves only from 0.989 to 1.000. The dense signal is therefore acting chiefly as a *re-ranking* prior — BM25 places the gold document in the candidate set, and the semantic signal promotes it to rank one. This is the mechanism by which the fusion earns its gain on this corpus.

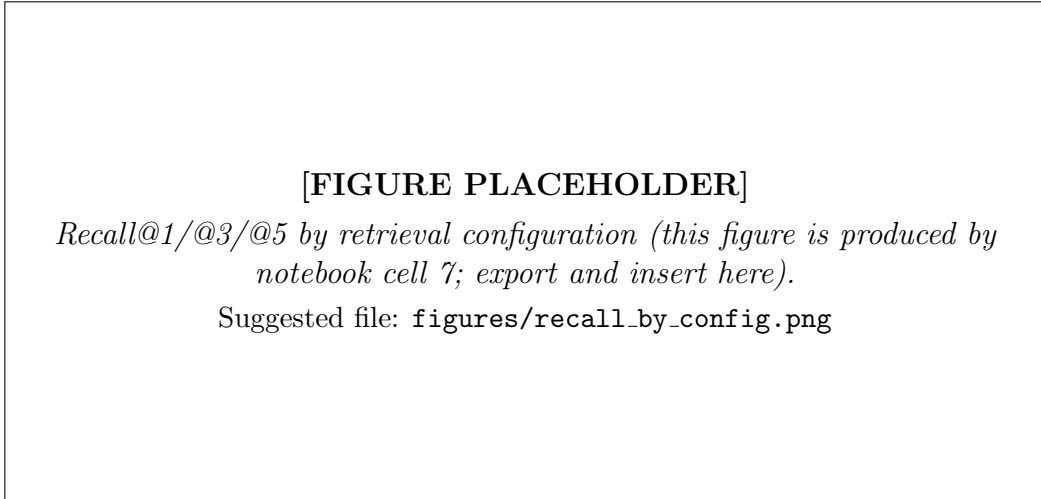


Figure 5: Recall@1/@3/@5 by retrieval configuration (this figure is produced by notebook cell 7; export and insert here).

6.2 Per-Class Analysis

The aggregate figures conceal the phenomenon of interest. Table 3 stratifies Recall@5 by query class and localizes the entire dense/sparse asymmetry.

Table 3: Recall@5 stratified by query class (source: notebook cell 6).

Query class	Dense	Sparse	Hybrid
id (identifier)	0.221	1.000	1.000
keyword	0.853	0.956	1.000
symptom	1.000	1.000	1.000
title	1.000	1.000	1.000

On **symptom** and **title** queries — paraphrastic and semantically rich — dense retrieval is already saturated at 1.000; the sparse component contributes nothing there, and critically, costs nothing either. On **identifier** queries the dense retriever collapses to 0.221. This is not a tuning deficiency but a consequence of the encoder’s inductive bias: all incident identifiers occupy a narrow region of the embedding space because they are semantically the same object, so cosine similarity cannot rank the correct one above its near-neighbors. BM25’s IDF term, which assigns near-maximal weight to a token appearing in exactly one document, resolves these exactly (1.000).

The hybrid attains 1.000 on all four classes. It does not merely average the components’ performance — it attains the upper envelope, because RRF requires only that *one*

constituent rank the gold document highly, and the two components’ failure modes do not overlap. Their complementarity is what makes the fusion effective; had both failed on the same queries, no rank fusion could recover them.

6.3 Ablations

Two parameters are ablated on the hybrid configuration (Table 4).

Table 4: Hyperparameter ablations, hybrid configuration (source: notebook cell 10).

(a) Chunk size		
Size (chars)	Chunks	Recall@5
400	334	1.000
700	180	1.000
1000	146	1.000
(b) Retrieved k		
k	Recall@5	MRR
1	0.974	0.974
3	1.000	0.987
5	1.000	0.987
10	1.000	0.987

Chunk size. Recall@5 is invariant at 1.000 across [400, 1000] characters, though the index size varies by a factor of 2.3 (334 to 146 chunks). The invariance follows from the corpus geometry: at a median of 147.5 tokens (Table 1), most reports are shorter than even the smallest window, so segmentation rarely splits a report’s discriminative content across chunks. Chunking would become a live hyperparameter on a corpus of longer documents; on this one it is not, and reporting it as insensitive is more useful than reporting a tuned value that does not generalize.

Retrieved k . Recall@5 saturates at $k = 3$ (0.974 at $k = 1$, 1.000 for $k \geq 3$). This licenses a design decision with direct cost consequences: the generation context is bounded to the top three reports rather than five. Since the marginal reports contribute no additional recall, the truncation is lossless with respect to retrieval while reducing prompt length — which, under local inference, is the dominant controllable term in generation cost. The ablation converts an arbitrary constant into a justified one.

6.4 Error Analysis

The hybrid configuration records **0 failures over 272 queries** (0.0%), with empty failure-by-class and failure-by-cause distributions (notebook cell 13). No query fails to place its gold report in the top five.

A zero error rate warrants scrutiny rather than celebration, and it should be read against the protocol’s stated bias (Section 4): with queries synthesized from the documents they retrieve, and a corpus of 68 documents rich in discriminative tokens, the task is easier than field retrieval. The result establishes that the retriever is not the limiting component under these conditions, and that the near-miss pairs deliberately embedded

in the corpus are correctly separated — but it does not establish a field error rate, and a harder benchmark would be required to differentiate configurations further. Notably, the aggregate is already saturated at Recall@5 for the hybrid, which means this benchmark has no remaining headroom to measure improvements against; this is itself a reason to construct a harder evaluation (Section 9).

7 Grounded Generation

7.1 Conditioning and Output Structure

Retrieved reports are formatted into a prompt with their similarity scores and identifiers, and the model is constrained to emit a structured JSON object: an incident classification, a confidence score, an ordered sequence of resolution steps each with a purpose, action, validation criterion, and evidence references, and typed supporting artifacts. Constrained JSON decoding removes conversational preamble and makes parsing deterministic. Generation length is capped, bounding worst-case latency.

Artifacts are typed by language rather than assumed. The correct artifact for an incident is domain-dependent — a shell command for a missing file, a manifest fragment for a container misconfiguration, a query for a data defect — and the model selects the language accordingly. Fixing the artifact type a priori (for instance, always emitting SQL) produces confidently-formatted but semantically irrelevant output on incidents outside that domain.

7.2 Abstention Under Insufficient Evidence

A generator instructed to always produce a resolution will produce one regardless of evidence, which converts absent information into fabricated content. The pipeline therefore admits abstention at two points.

First, before retrieval, an intent classifier routes each turn. Inputs that are conversational, or that are incident-flavored but carry no diagnosable content (no error code, identifier, file path, service name, or quoted message), are answered with a request for specifics. Because this precedes both retrieval and generation, fabrication on under-specified input is structurally impossible rather than discouraged. Follow-up turns are exempt, as prior dialogue supplies the missing context.

Second, the generation prompt permits the model to declare insufficient evidence and lower its confidence rather than emit a fixed number of steps, and forbids citing identifiers absent from the retrieved set. The system also distinguishes a model backend failure from a low-confidence answer: an unreachable model raises an explicit error rather than returning a well-formed but empty result, which would otherwise be indistinguishable from a genuine negative diagnosis.

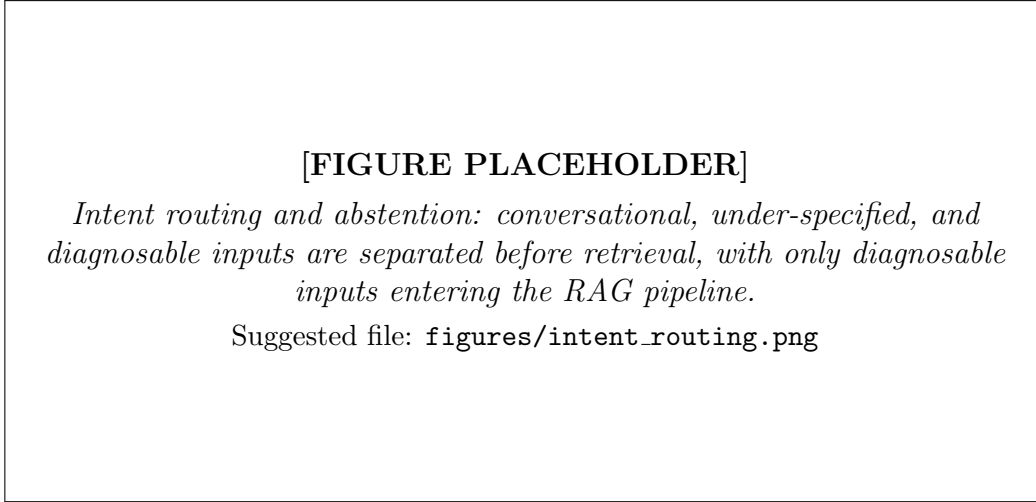


Figure 6: Intent routing and abstention: conversational, under-specified, and diagnosable inputs are separated before retrieval, with only diagnosable inputs entering the RAG pipeline.

7.3 Adversarial Input

User text is screened for instruction-override and prompt-extraction patterns; detected inputs are enclosed in an explicit untrusted-data delimiter within the prompt, so the model treats them as content to analyze rather than instructions to follow, and the system prompt is never emitted. A legitimate incident containing such phrasing is still processed as an incident, with the detection surfaced to the user, rather than refused. Secrets and personal data (credentials, bearer tokens, connection strings, private keys, email addresses) are redacted from dialogue text before persistence.

7.4 Feedback

Answers may be rated, and a negative rating admits a free-text correction. Corrections are stored and, on subsequent queries with lexical overlap to the corrected question, injected into the prompt as high-priority guidance. The mechanism is explicitly a retrieval-time intervention over an inspectable store, not parameter adaptation: no model weights are modified, and the system makes no claim to the contrary.

8 Report Synthesis

The corpus is the system’s memory, so its growth determines long-term utility. Because the assistant’s output and the corpus documents share the schema of Section 3, a resolved dialogue can be projected directly onto a report, inverting the ingestion projection π .

The synthesis maps dialogue elements onto schema fields under a strict grounding rule: the reported symptom is taken from the initiating query, the root cause from the model’s diagnosis, the resolution steps from the generated step sequence, code blocks from the typed artifacts, and cross-references from the cited reports. Fields not determined by the dialogue are left empty rather than filled by inference, and identifying metadata records provenance. Every populated value is therefore traceable to dialogue content.

The synthesized report is validated against the same schema as authored reports and persisted through the same service, so it is indistinguishable downstream: it is editable in the report editor and, once written, becomes retrievable corpus. This closes the loop between the system’s two modules — the assistant consumes the corpus to resolve incidents and extends it with each resolution.

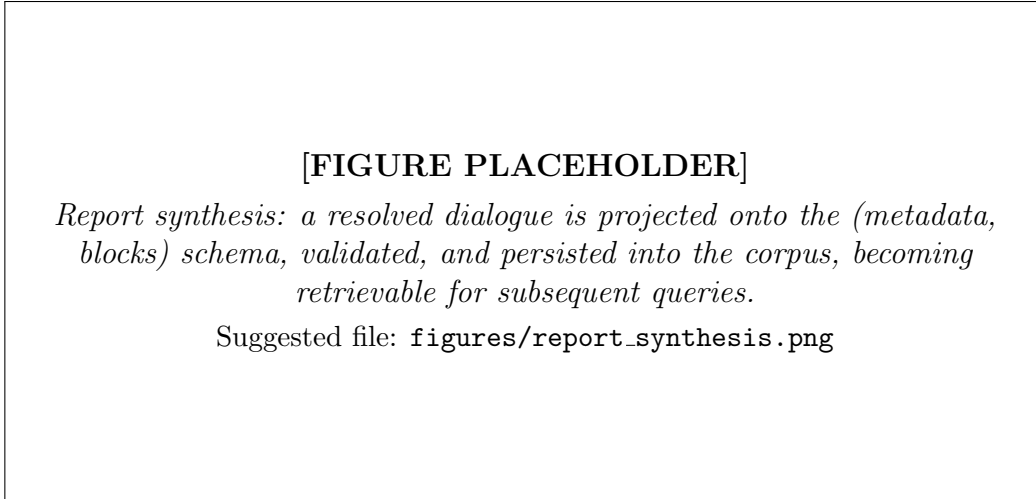


Figure 7: Report synthesis: a resolved dialogue is projected onto the (metadata, blocks) schema, validated, and persisted into the corpus, becoming retrievable for subsequent queries.

9 Discussion

9.1 Interpretation

The central result is the per-class decomposition of Table 3. It shows that the hybrid’s aggregate advantage is not a uniform improvement but the union of two disjoint competencies: the dense component contributes on paraphrastic queries where it is already saturated, and the sparse component contributes on identifier queries where the dense component fails almost completely (0.221). The fusion attains the upper envelope precisely because the failure modes are disjoint — a property of this query distribution, not a universal one.

The secondary result is methodological. Both the representation analysis (Section 4) and the k -ablation (Section 6.3) indicate that this system’s quality is governed by *what is retrieved and how it is represented*, not by generative capacity. Under a constraint that caps model scale, this is the actionable conclusion: the tractable levers are the projection π , the retrieval method, and the context bound.

9.2 Limitations

Protocol validity. Queries are synthesized from the documents they retrieve. This measures recovery from self-paraphrase and yields an upper bound on field performance; real queries use vocabulary absent from the report and may under-specify. The comparative conclusions are more robust than the absolute values, since all configurations are measured under the same bias.

Benchmark saturation. The hybrid attains $\text{Recall@5} = 1.000$ and zero errors, leaving no headroom to distinguish further improvements. The benchmark has served its discriminative purpose but cannot support finer comparison.

Corpus scale. With 68 documents, the retrieval task is comparatively easy and the IDF statistics that make BM25 effective are estimated from few documents. Behavior at larger corpus scale — where dense retrieval’s approximate-nearest-neighbor regime and BM25’s term statistics both change — is not characterized here.

Unmeasured generation. The evaluation covers retrieval only. Generation quality and end-to-end latency are not quantified, and no claims about them are made.

9.3 Future Work

The natural extensions follow from the limitations. A benchmark of real queries with human relevance judgments would establish field validity and restore measurement headroom. Evaluating generation — faithfulness to retrieved evidence, and correctness of proposed resolutions — would characterize the component this work leaves unmeasured. Learning a re-ranker from accumulated feedback would replace the current retrieval-time correction injection with a parametric model. Finally, scaling the corpus by an order of magnitude would test whether the per-class asymmetry observed here persists.

9.4 Conclusion

This work designs and evaluates a locally-hosted retrieval-augmented incident assistant whose retrieval stage fuses dense and sparse indices by Reciprocal Rank Fusion. On a 272-query benchmark, the hybrid attains $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense-only retrieval, with the gain localized by per-class analysis to identifier queries where dense retrieval attains 0.221. Ablations establish invariance to chunk size on this corpus and saturation at $k \geq 3$, which bounds the generation context without recall loss. The retriever is embedded in a generation pipeline that abstains under insufficient evidence rather than fabricating, and in a synthesis mechanism that returns resolved dialogues to the corpus as validated structured documents. The system satisfies its privacy constraint throughout, executing all inference locally.

References

- [1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. In *Proceedings of the 32nd ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759.
- [3] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In *Proceedings of EMNLP-IJCNLP*, 3982–3992.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In *Advances in Neural Information Processing Systems* 33, 9459–9474.

- [5] Karpukhin, V., Oğuz, B., Min, S., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. In Proceedings of EMNLP, 6769–6781.
- [6] Järvelin, K., & Kekäläinen, J. (2002). *Cumulated Gain-Based Evaluation of IR Techniques*. ACM Transactions on Information Systems, 20(4), 422–446.